Introduction à l'IA - TP SAT

Nom : Hassani Ghizlane Groupe : TIPS Date : 04/06/2026

Exercice 1 :

Formalisation

On choisit quatre variables propositionnelles correspondant aux faits de l'énoncé :

- B : « vous possédez un badge »
- A : « vous êtes accompagné »
- E : « vous pouvez entrer dans la salle »
- D : « l'alarme est désactivée »

Les phrases de l'énoncé se traduisent par les formules logiques suivantes :

- Si badge alors entrer : $B \rightarrow E$
- Si accompagne alors entrer : $A \rightarrow E$
- Si on entre alors alarme désactivée : $E \rightarrow D$
- L'alarme est activée! (donc PAS désactivée) : $\neg D$
- On possède un badge ET on est accompagné : $B \wedge A$

Mise en Forme Normale Conjonctive (FNC / CNF)

On élimine les implications avec la règle $X \rightarrow Y \equiv \neg X \vee Y$, ce qui donne la conjonction de clauses :

$$F = (\neg B \vee E) \wedge (\neg A \vee E) \wedge (\neg E \vee D) \wedge (\neg D) \wedge (B) \wedge (A)$$

Codage entier des variables pour le solveur : B = 1, A = 2, E = 3, D = 4 (un littéral négatif représente la négation).

Avec le codage B = 1, A = 2, E = 3 et D = 4, les clauses que je donne au solveur sont : [-1, 3], [-2, 3], [-3, 4], [-4], [1] et [2].

1 - Résultat obtenu

Quand je lance le solveur, il me répond que la formule n'est pas satisfiable. En y réfléchissant c'est normal : comme B est vrai, E doit l'être aussi (à cause de B → E), donc D également (à cause de E → D), mais la clause ¬D dit exactement le contraire. Les deux ne peuvent pas tenir en même temps.

Pour avoir une formule satisfiable, j'ai modifié une clause : j'ai remplacé ¬D (alarme activée) par D (alarme désactivée), ce qui me semble plus cohérent quand on entre dans la salle. Cette fois le solveur trouve une solution, avec B, A, E et D tous vrais.

```
Encodage direct, satisfiable ? False
Apres modification, satisfiable ? True
Modele : [1, 2, 3, 4]
Tseitin sur F, satisfiable ? False
```

2 - Transformation de Tseitin sur F

Le principe de Tseitin, c'est de donner un nom (une nouvelle variable) à chaque sous-formule, puis d'écrire l'équivalence t comme variable ↔ correspondante sous forme de clauses. J'utilise ces trois modèles :

- $t \leftrightarrow (a \rightarrow b)$: $(\neg t \vee \neg a \vee b) \wedge (t \vee a) \wedge (t \vee \neg b)$
- $t \leftrightarrow (a \wedge b)$: $(\neg t \vee a) \wedge (\neg t \vee b) \wedge (t \vee \neg a \vee \neg b)$
- $t \leftrightarrow \neg a$: $(\neg t \vee \neg a) \wedge (t \vee a)$

Variables auxiliaires (codage entier a la suite des variables de base) :

Variable	Définition
$x1 = 5$	$B \rightarrow E$
$x2 = 6$	$A \rightarrow E$
$x3 = 7$	$E \rightarrow D$
$x4 = 8$	$\neg D$
$x5 = 9$	$x1 \wedge x2$
$x6 = 10$	$x5 \wedge x3$
$x7 = 11$	$x6 \wedge x4$
$x8 = 12$	$x7 \wedge B$
$x9 = 13$	$x8 \wedge A \ (= racine\ F)$

J'introduis donc $x1 = B \rightarrow E$, $x2 = A \rightarrow E$, $x3 = E \rightarrow D$ et $x4 = \neg D$, puis je regroupe les conjonctions petit à petit : $x5 = x1 \wedge x2$, $x6 = x5 \wedge x3$, $x7 = x6 \wedge x4$, $x8 = x7 \wedge B$, et enfin $x9 = x8 \wedge A$ qui représente toute la formule F. Je code ces variables de 5 à 13 et j'impose la racine [13]. Voici les clauses obtenues :

```
x1 = B→E     : [-5,-1,3]  [5,1]  [5,-3]
x2 = A→E     : [-6,-2,3]  [6,2]  [6,-3]
x3 = E→D     : [-7,-3,4]  [7,3]  [7,-4]
x4 = ¬D      : [-8,-4]   [8,4]
x5 = x1∧x2   : [-9,5]   [-9,6]   [9,-5,-6]
x6 = x5∧x3   : [-10,9]  [-10,7]  [10,-9,-7]
x7 = x6∧x4   : [-11,10] [-11,8]  [11,-10,-8]
x8 = x7∧B    : [-12,11] [-12,1]  [12,-11,-1]
x9 = x8∧A    : [-13,12] [-13,2]  [13,-12,-2]
racine       : [13]
```

Résultat : Après avoir ajouté toutes ces clauses et lancé le solveur, j'obtiens encore une fois insatisfiable, comme avec l'encodage direct. C'est ce que j'attendais, parce que la transformation de Tseitin ne change pas la satisfiabilité de la formule.

Exercice 2 :

$\Phi \ = \ ((P \to Q) \wedge (R \vee S)) \to (P \wedge R)$

On décompose la formule en sous-formules, en introduisant une variable auxiliaire pour chacune. Codage : P = 1, Q = 2, R = 3, S = 4

- $a \ = \ 5 \leftrightarrow \ P \to Q$
- $b \ = \ 6 \leftrightarrow \ R \vee S$
- $c \ = \ 7 \leftrightarrow \ a \wedge b \ \ = \ (P \to Q) \wedge (R \vee S)$
- $d \ = \ 8 \leftrightarrow \ P \wedge R$
- $t \ = \ 9 \ \leftrightarrow \ c \to d \ \ = \ \Phi \ \ (racine)$

Clauses obtenues pour chaque equivalence (puis on impose la racine [9]) :

```
a = P→Q   : [-5,-1,2]  [5,1]  [5,-2]
b = R∨S   : [-6,3,4]   [6,-3] [6,-4]
c = a∧b   : [-7,5]  [-7,6]  [7,-5,-6]
d = P∧R   : [-8,1]  [-8,3]  [8,-1,-3]
t = c→d   : [-9,-7,8]  [9,7]  [9,-8]
racine   : [9]
```

Résultat obtenu

Le solveur répond SATISFIABLE (True), avec par exemple le modèle P vrai, Q faux, R faux, S faux.

Interprétation : il suffit que l'antécédent ((P→Q) ∧ (R∨S)) soit faux pour que l'implication Φ soit vraie. Ici P vrai et Q faux rendent (P→Q) faux, donc l'antécédent faux, donc Φ est vraie. La formule possède bien un modèle : elle est satisfiable.

```
Satisfiable ? True
Modele : [1, -2, -3, -4, -5, -6, -7, -8, 9]
P = True  Q = False  R = False  S = False
```

Exercice 3 :

1 - Construction du graphe

Chaque produit est un sommet ; chaque incompatibilité est une arête. Les incompatibilités sont : $p1 - p2,\ p1 - p3,\ p3 - p4,\ p4 - p2$.

En les dessinant, je me rends compte que ça forme un cycle à quatre sommets : $p1 - p2 - p4 - p3 - p1$.

2 - Nombre de conteneurs

Comme c'est un cycle de longueur paire, deux couleurs suffisent (le graphe est biparti) : il me faut donc 2 conteneurs. Par exemple, je mets p1 et p4 ensemble, puis p2 et p3 ensemble, et chaque arête relie bien deux couleurs différentes. Un seul conteneur est impossible puisqu'il y a des arêtes.

3 - Variables propositionnelles

Je prends une variable par couple (produit, couleur) : x(i, c) signifie « le produit pi est dans le conteneur c ». Avec 4 produits et 2 couleurs, cela fait 8 variables. Je les numérote avec

$$v(i,\ c)\ =\ (i\ -\ 1) \times nb_couleurs\ +\ c.$$

Un littéral positif veut dire que le produit a cette couleur, un littéral négatif qu'il ne l'a pas. Du coup, une situation complète (une solution) se lit ainsi : pour chaque produit, une seule de ses variables couleur est vraie et toutes les autres sont fausses.

4 - Modélisation des contraintes

Chaque produit doit avoir exactement une couleur, c'est-à-dire au moins une ET au plus une (on ne peut ni ne pas en avoir, ni en avoir plusieurs).

Trois familles de contraintes, exprimées en clauses :

- Au moins une couleur par produit : pour chaque i, $(x(i, 1) \vee x(i, 2))$.
- Au plus une couleur par produit : pour chaque i et chaque paire de couleurs c1 < c2, $(\neg x(i, c1) \vee \neg x(i, c2))$.
- Produits incompatibles de couleurs différentes : pour chaque arête (i,j) et chaque couleur c, $(\neg x(i, c) \vee \neg x(j, c))$.

5 - Implémentation et résultat

J'ai d'abord testé avec une seule couleur, et le solveur répond insatisfiable. Avec deux couleurs il trouve une solution : p1 et p4 dans le conteneur 1, p2 et p3 dans le conteneur 2. Ça correspond exactement à ce que j'avais trouvé à la main à la question 2.

```
Avec 1 couleur(s) : satisfiable ? False
Avec 2 couleur(s) : satisfiable ? True
   p1 -> conteneur 1
   p2 -> conteneur 2
   p3 -> conteneur 2
   p4 -> conteneur 1
```

Exercice 4 :

1 - Nombre de variables

J'ai 4 lignes, 4 colonnes et 4 chiffres possibles. En mettant une variable par combinaison (i, j, k), j'obtiens 4 × 4 × 4 = 64 variables.

2 - Variable propositionnelle

On choisit :

x(i, j, k) = « la case (ligne i, colonne j) contient le chiffre k »

Codage entier (numero unique de 1 a 64) :

$$v(i,\ j,\ k)\ =\ (i\ -\ 1) \times 16\ +\ (j\ -\ 1) \times 4\ +\ k$$

3 - Une clause par contrainte (exemples)

- la case (1,4) contient au moins une valeur : $x(1, 4, 1) \vee x(1, 4, 2) \vee x(1, 4, 3) \vee x(1, 4, 4)$
- la case (1,4) contient au plus une valeur (ex. les chiffres 1 et 2) : $\neg x(1, 4, 1) \vee \neg x(1, 4, 2)$
- le chiffre 1 apparaît dans la ligne 1 : $x(1, 1, 1) \vee x(1, 2, 1) \vee x(1, 3, 1) \vee x(1, 4, 1)$
- le chiffre 1 apparaît dans la colonne 1 : $x(1, 1, 1) \vee x(2, 1, 1) \vee x(3, 1, 1) \vee x(4, 1, 1)$
- le chiffre 1 apparaît dans le bloc en haut à gauche:
  $x(1, 1, 1) \vee x(1, 2, 1) \vee x(2, 1, 1) \vee x(2, 2, 1)$

4 - Génération des clauses en Python

On parcourt toutes les valeurs avec des boucles et la fonction itertools.combinations pour les contraintes « au plus un » :

```python
def v(i, j, k):
    return (i - 1) * N * N + (j - 1) * N + k

def generer_clauses():
    clauses = []
    ch = range(1, N + 1)
    for i in range(1, N + 1):
        for j in range(1, N + 1):
            clauses.append([v(i, j, k) for k in ch])
            for k1, k2 in combinations(ch, 2):
                clauses.append([-v(i, j, k1), -v(i, j, k2)])
    for k in ch:
        for i in range(1, N + 1):
            clauses.append([v(i, j, k) for j in range(1, N + 1)])
            for j1, j2 in combinations(range(1, N + 1), 2):
                clauses.append([-v(i, j1, k), -v(i, j2, k)])
        for j in range(1, N + 1):
            clauses.append([v(i, j, k) for i in range(1, N + 1)])
            for i1, i2 in combinations(range(1, N + 1), 2):
                clauses.append([-v(i1, j, k), -v(i2, j, k)])
    # blocs 3x3
    for bi in range(0, N, 3):
        for bj in range(0, N, 3):
            cases = [(bi + di, bj + dj) for di in (1, 2, 3) for dj in (1, 2, 3)]
            for k in ch:
                clauses.append([v(i, j, k) for (i, j) in cases])
                for a, b in combinations(cases, 2):
                    clauses.append([-v(a[0], a[1], k), -v(b[0], b[1], k)])
    return clauses
```

5 - Ajouter des valeurs initiales

Pour fixer une case déjà remplie, j'ajoute simplement une clause unitaire. Par exemple « la case (1,4) vaut 4 » devient la clause [v(1,4,4)]. Je fais ça pour chaque indice connu de la grille :

```
indices = {(1, 4): 4, (3, 1): 2, (3, 4): 3, (4, 1): 4, (4, 3): 1, (4, 4): 2}
for (i, j), k in indices.items():
    s.add_clause([v(i, j, k)])
```

6 - Interpréter et reconstruire la grille

Le modèle renvoyé par le solveur est une liste de littéraux. Un littéral positif v(i,j,k) veut dire que la case (i,j) contient le chiffre k. Je parcours donc les littéraux positifs pour reconstruire la grille :

```
def reconstituer_grille(modele):
    positifs = set(x for x in modele if x > 0)
    grille = [[0] * N for _ in range(N)]
    for i in range(1, N + 1):
        for j in range(1, N + 1):
            for k in range(1, N + 1):
                if v(i, j, k) in positifs:
                    grille[i - 1][j - 1] = k
    return grille
```

Résultat obtenu pour la grille de la figure 1 :

```
[1, 2, 3, 4]
[3, 4, 2, 1]
[2, 1, 4, 3]
[4, 3, 1, 2]
```

Bonus - Sudoku 9 x 9

J'ai repris le même modèle en passant N à 9 et la taille des blocs à 3, ce qui donne 9 × 9 × 9 = 729 variables. Le code est presque identique, et le solveur résout une grille classique sans problème.

```
[5, 3, 4, 6, 7, 8, 9, 1, 2]
[6, 7, 2, 1, 9, 5, 3, 4, 8]
[1, 9, 8, 3, 4, 2, 5, 6, 7]
[8, 5, 9, 7, 6, 1, 4, 2, 3]
[4, 2, 6, 8, 5, 3, 7, 9, 1]
[7, 1, 3, 9, 2, 4, 8, 5, 6]
[9, 6, 1, 5, 3, 7, 2, 8, 4]
[2, 8, 7, 4, 1, 9, 6, 3, 5]
[3, 4, 5, 2, 8, 6, 1, 7, 9]
```